Recent Kolmogorov–Arnold Network (KAN) Architecture -level developments

---

Purpose and Scope

Note : no paper using KAN in single-molecule localization microscopy, or PSF-based emitter localization was found in this window. This is consistent with the KAN-SMLM direction remaining a novel, unclaimed intersection as of this search.

Papers covered:

- Faroughi, Mostajeran, Mashhadzadeh & Faroughi — "Kolmogorov-Arnold networks for data-driven, physics-informed, and deep-operator learning: a review, synthesis, and new analysis," Neural Networks 200 (2026) 108791.
- Zhang, Fan & Lu — "RSS-KAN: Replacing B-splines with ReLU, Sigmoid, and Sine for accurate and efficient Kolmogorov–Arnold Networks," Neurocomputing 672 (2026) 132796.
- Wolff, Alesiani, Duhme & Jiang — "Clifford Kolmogorov-Arnold Networks," arXiv:2602.05977v2 (revised June 23, 2026).
- James Bagrow and Josh Bongard, "Optimized Architectures for Kolmogorov-Arnold Networks," Apr 20, 2026.

1. Faroughi et al. — KAN Review, Synthesis, and New Analysis

DOI: https://doi.org/10.1016/j.neunet.2026.108791

The core idea of a KAN, as explained in the paper

A standard neural network (MLP) puts the learnable part on the connections between neurons as fixed weights, and puts a fixed, non-learnable nonlinearity (like ReLU) at each neuron. A KAN flips this: the connections carry learnable curved functions instead of fixed weights, and the neurons just add things up. instead of learning how much to weight each input, a KAN learns the actual shape of the function applied to each input before those results are summed. The paper frames this as more mathematically grounded, since it directly mirrors a classical theorem (the Kolmogorov-Arnold theorem) about representing any multi-variable function as a sum of simpler single-variable functions. this shape-based structure is also why KANs are described as more interpretable than MLPs: you can look at each edge's learned curve and see what it's doing, rather than reading an opaque weight matrix.

The original KAN implementation represents each edge's learned curve using B-splines (smooth, piecewise polynomial curves, the same kind of curve used in CAD software or font design). Later variants, discussed throughout the review, swap this B-spline curve for other curve types — Chebyshev polynomials, wavelets, radial basis functions (bell-shaped bumps) — each with different trade-offs in smoothness, parameter count, and how well they capture sharp or oscillatory behavior.

A KAN can also be stacked in layers just like an MLP, and the paper notes that shallow (2-layer) KANs, while mathematically the "pure" form of the underlying theorem, are often not expressive enough in practice, so most modern KANs use multiple layers of these learned-curve connections.

Data-driven KANs — how they're trained and what the comparisons showed

In the purely data-driven setting, a KAN is just trained like any regression model: feed it inputs, compare its output to the true output, and adjust the learned curves to reduce the error. The paper's own new experiments used a KAN with Chebyshev-polynomial edges (called cKAN) to predict how a granular material (like sand) deforms under stress, comparing it against a matched MLP.

Reported takeaways from that comparison:

- The KAN version was noticeably more accurate at predicting stress under a loading path it had never seen during training, with roughly 3 to 6 times lower error than the MLP depending on the exact setup.
- During training, the KAN's error dropped steadily and kept improving, while the MLP's error dropped fast at first and then plateaued early — described as the KAN continuing to learn where the MLP stalls.
- The KAN version took somewhat longer per training step than the MLP (roughly 50-60% more time per epoch), because each learned curve is more expensive to evaluate than a simple weighted sum. So the KAN's edge in this experiment was pure accuracy and continued learning, not raw speed.

Physics-informed KANs (PIKANs)

A PIKAN is a KAN that's trained not just to match data, but to also satisfy a known physics equation (a PDE) as an extra training objective — the same idea as a Physics-Informed Neural Network (PINN), just with a KAN instead of an MLP doing the learning. The network is fed spatial and time coordinates and asked to output a physical quantity like pressure, displacement, or temperature; during training, the model's own output is checked against the governing equation, and any violation of that equation is penalized just like a normal prediction error would be.

A key structural choice explored is how strictly to enforce boundary and initial conditions. In a "soft" version, violating the boundary condition is just penalized as part of the loss, so the network is only encouraged, not guaranteed, to get it right. In a "hard" version, the network's output is mathematically wrapped so it automatically satisfies the boundary condition no matter what the network predicts internally — a structural guarantee rather than a training incentive.

The paper's central practical finding here is about basis-function choice: switching a PIKAN's edge curves from B-splines to Chebyshev polynomials, with no other change, cut the error by roughly 90% on a benchmark PDE (the Allen-Cahn equation). Pairing that Chebyshev-based PIKAN with a more advanced optimizer (a quasi-Newton method called SSBroyden, instead of the usual Adam) pushed the error down by another 99% on top of that. Across the many comparisons collected in the review, PIKANs are generally reported to beat equivalent PINNs by 85-99% in error, and usually with fewer trainable parameters, not more.

The paper also describes a scaling trick (rescaling the input domain and the physics-residual terms to fit the natural range of the Chebyshev curves) that made a big difference on a harder, wider version of the Allen-Cahn problem — an 88-90% error reduction just from that rescaling step, with no architecture change.

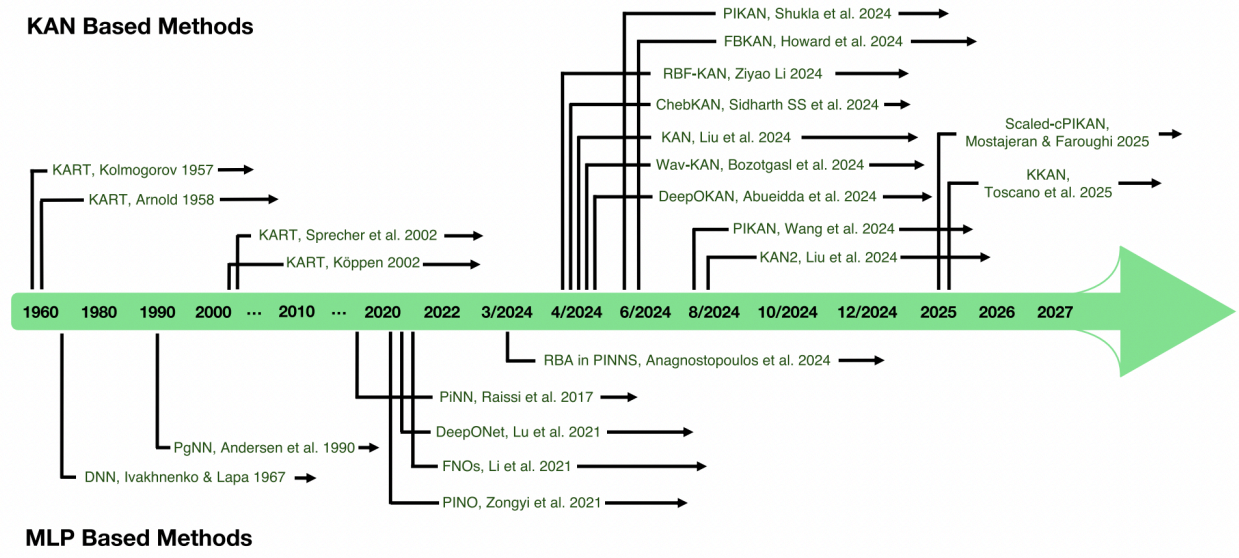
Deep-operator KANs (DeepOKANs)

A normal PIKAN learns one specific solution for one specific set of boundary/initial conditions. A DeepOKAN instead learns an entire family of solutions at once — given a new initial condition or boundary condition it hasn't seen, it can produce the right solution without being retrained. Structurally, this is done with two sub-networks: a "branch" network that encodes the specific input condition (e.g., a crack shape, or an initial temperature profile), and a "trunk" network that encodes the query point (where in space/time you want the answer). Their outputs are combined to produce the final prediction. DeepOKAN is simply this same two-network structure, but built from KAN layers instead of MLP layers in both branches.

Reported comparisons against the MLP-based version of this idea (DeepONet) showed the KAN-based version was substantially more accurate in shallow (simple) architectures — over an 85% error reduction on a flow-through-porous-media benchmark — though the gap narrowed once both models were made deeper. The KAN version also held up noticeably better when noise was added to the input data, whereas the MLP version's accuracy degraded faster as noise increased.

Stated limitations of KANs, as discussed in the review

- KANs are reported to struggle, relative to MLPs, on functions with sharp jumps or discontinuities, and to be more sensitive to noisy data.
- Training a KAN can take substantially longer than training an MLP of similar size — the paper cites cases where it took up to two orders of magnitude (100x) longer, particularly for the original B-spline version.
- KANs perform worse than MLPs when there isn't much training data available.
- A structural/engineering complaint: KANs aren't natively supported by mainstream deep learning libraries the way MLPs are, and the original B-spline curve evaluation involves a step-by-step (recursive) calculation that doesn't parallelize well on GPUs.



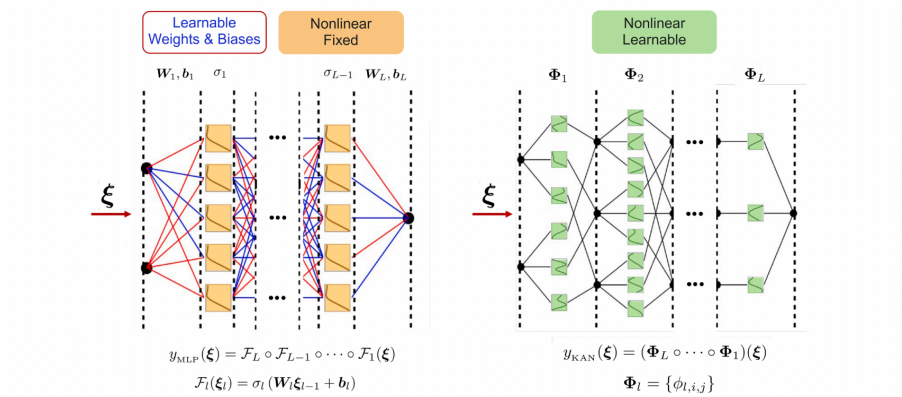


Fig. 2. A conceptual illustration of how MLPs and KANs work. MLPs use a fully interconnected structure that mixes all input dimensions in each layer; in contrast, KANs process each dimension with a univariate function, then combine the results via summation. This dimension-wise approach can offer a more direct path to representing high-dimensional functions. Here, $\boldsymbol{\xi}$ represents the input vector, $\boldsymbol{W}_{(l)}$ and $\boldsymbol{b}_{(l)}$ refer to the weights and biases of the l-th layer, respectively, σ is a fixed activation function that can take different forms, e.g., ReLU, sigmoid, sin, tanh, Faroughi et al. (2023), and $\boldsymbol{\Phi}_l$ is a KAN layer defined as a matrix of 1-dimensional functions.

Table 1
Comparison of basis functions used in KANs with N_l layers, N_n neurons per layer, grid size g, and polynomial degree k. The table outlines their parameterization, computational order, and suitability for function approximation.

Basis Function	Parameters θ	Order	Applications	Ref
Splines	$\{w_s, w_b, c_i\}$	$O(N_l N_n^2(k+g))$	Robust for structured data with local control; high parameter count for complex functions	(Bohra et al., 2020; Li, 2024; Samadi et al., 2024; Vaca-Rubio et al., 2024)
Chebyshev	$\{c_i\}$	$O(N_l N_n^2 k)$	Efficient for smooth functions; fewer parameters and fast convergence; requires input normalization	(Boyd, 2001; Guo et al., 2024; Hu et al., 2024; Mason & Handscomb, 2002; Mostajeran & Faroughi, 2025; SS et al., 2024)
Wavelets	$\{d_{\ell,r}, c_{\ell,r}, \sigma_{\ell,r}\}$	$O(3N_l N_n^2)$	Suitable for functions with sharp, high-amplitude oscillations; ideal for multi-resolution data	(Ali & Aly, 2024; Bhat et al., 2024; Bozorgasl & Chen, 2024; Daubechies, 1992; Gauthier et al., 2022; Ilyas et al., 2024; Levie et al., 2021; Mallat, 1999; Mostajeran & Hosseini, 2023a; Ramalakshmi et al., 2024; Torrence & Compo, 1998)
RBFs	$\{w_r, c_r, \sigma_r\}$	$O(3N_l N_n^2)$	Effective for smooth interpolation; compact, adaptive, and easy to tune with kernel variations	(Bouzidi et al., 2025; Buhmann, 2003; Ku et al., 2024; Liu et al., 2025a,b; Mirzaei & Nazemi, 2024; Mostajeran & Hosseini, 2023b; Muthusamy et al., 2025; Stenkin & Gorbachenko, 2024; Wendland, 2004)

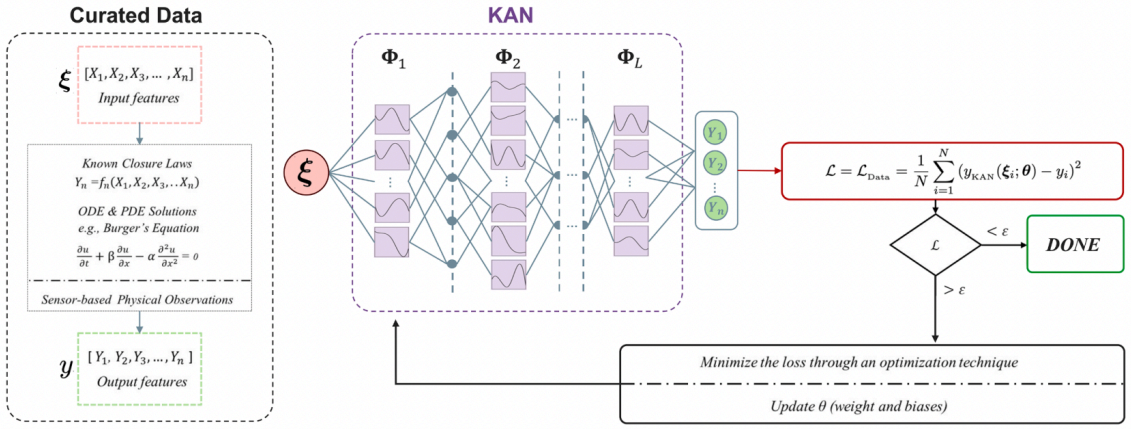


Fig. 3. Schematic architecture of a data-driven KAN model. The network receives input features and directly approximates the target output by learning a parameterized mapping $y_{\text{KAN}}(\boldsymbol{\xi};\boldsymbol{\theta})$ from data, without using physical governing equations. It employs a modified feed-forward architecture with learnable activation functions. The parameters $\boldsymbol{\theta}$ are optimized by minimizing a supervised loss, typically the mean squared error, using gradient-based methods, enabling the network to capture the underlying structure in the observed data.

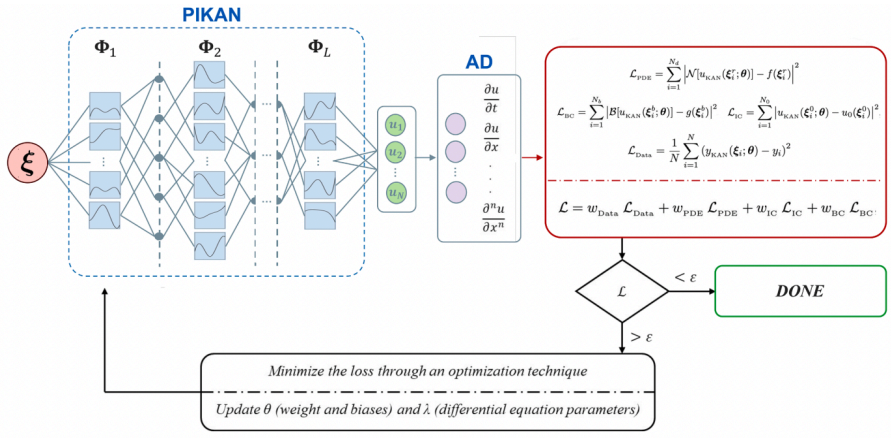


Fig. 7. Schematic architecture of PIKANs. The network receives spatiotemporal coordinates as input and approximates the multiphysics solution u (forward modeling) as well as unknown parameters in the governing equation (inverse modeling). The final layer computes derivatives of u with respect to input variables, which are used to formulate the residuals of the strong form of governing equations and data loss terms, each weighted by distinct coefficients. The learnable parameters $\boldsymbol{\theta}$ (neural weights and biases) and λ (unknown parameters) are optimized simultaneously through loss minimization.

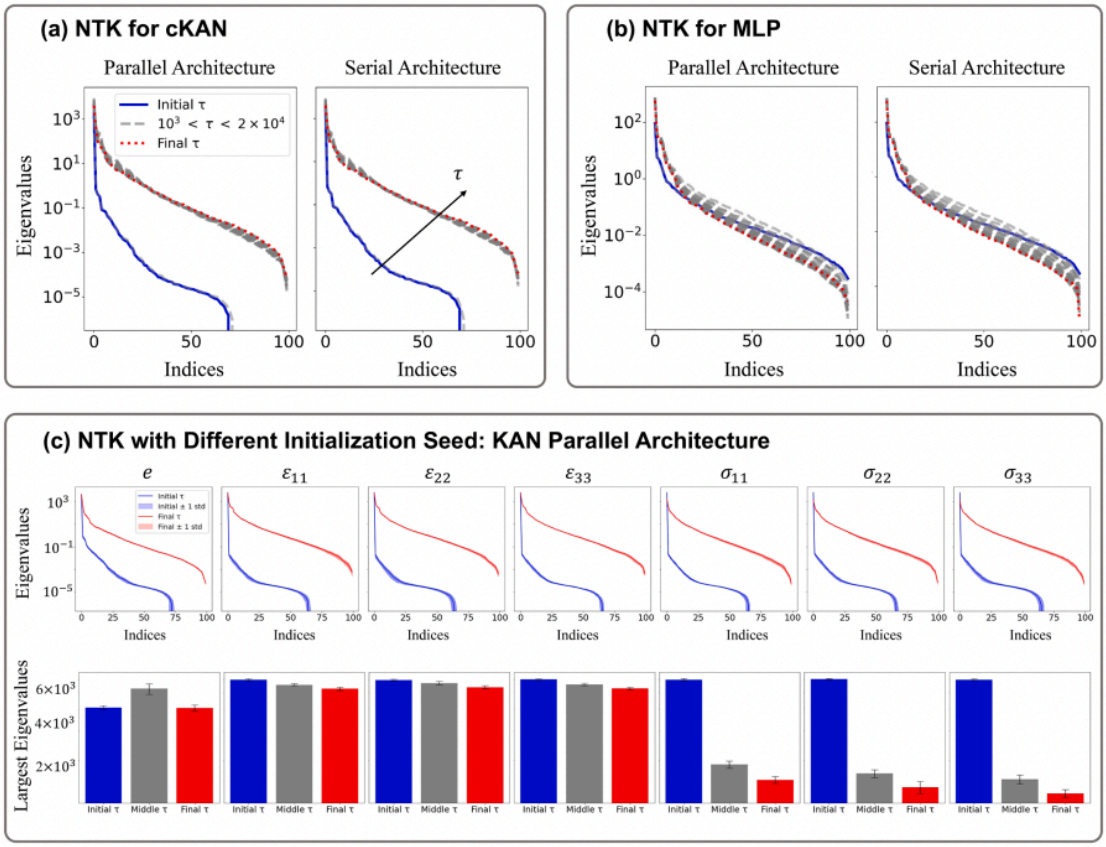


Fig. 6. Evolution of NTK eigenvalues during training of the void ratio subnetwork e, using parallel and serial architectures introduced in Mostajeran and Faroughi (2024). **(a)** cKAN architectures. They exhibit a more stable spectral distribution. **(b)** MLP architectures. They show less structured and more dispersed eigenvalue spectra. **(c)** Evaluation of the NTK matrices across multiple random initializations (10 seeds). The narrow standard-deviation bands indicate that both the initial and final NTK spectra are highly stable with respect to random initialization. All panels present new analysis based on Mostajeran and Faroughi (2024).

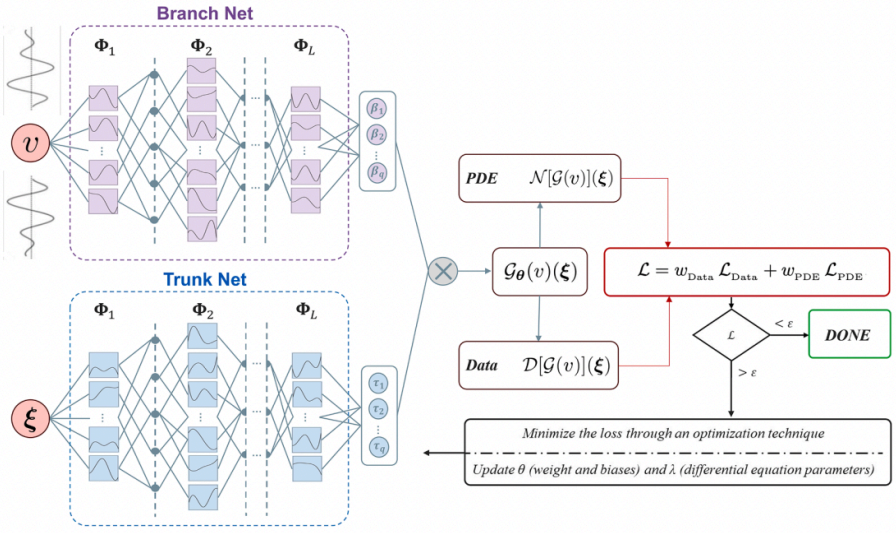


Fig. 15. A schematic architecture of DeepOKANs composed of a branch network that encodes samples of the input functions, extracting their latent representations, and a trunk network that extracts latent representations of spatiotemporal points in the solution domain at which the output functions are evaluated. In this setup, a dot product is then used to merge the latent representations extracted by each subnetwork and obtain a continuous and differentiable representation of the output functions. In a pure data-driven format $\lambda = 0$. In a physics-informed format, the data loss also contains the initial and boundary condition loss terms, and automatic differentiation is used to formulate appropriate terms in both data and PDE loss expressions.

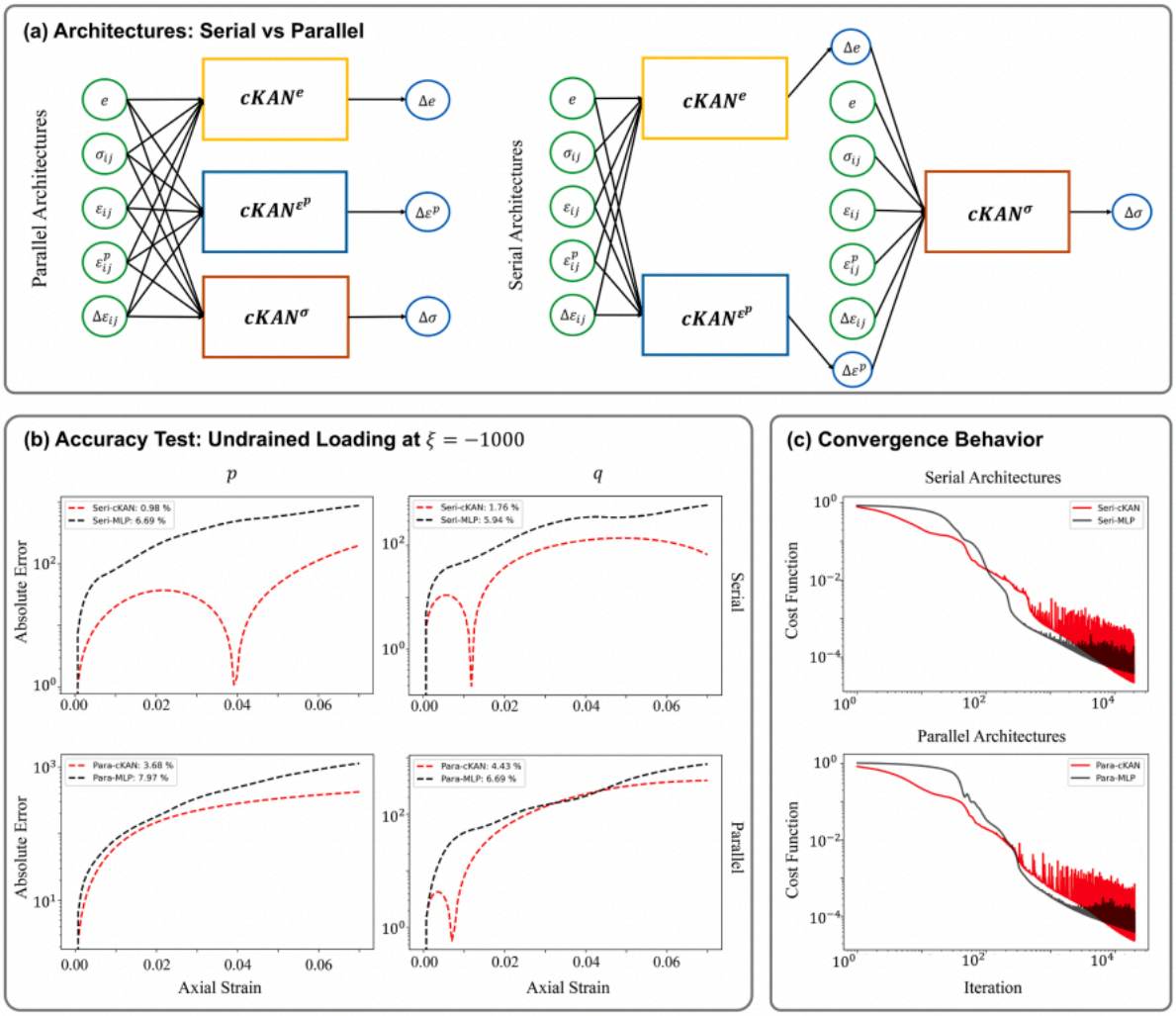


Fig. 5. Performance comparison of cKAN and MLP architectures for data-driven modeling of sand elasto-plasticity.**(a)** Network architectures (serial and parallel). The models take as input a vector of stress $\boldsymbol{\sigma}$, strain $\boldsymbol{\epsilon}$, plastic strain $\boldsymbol{\epsilon}^{\mathbf{p}}$, void ratio e, and the incremental strain $\Delta\boldsymbol{\epsilon}$, and the output captures the corresponding changes in material response, including changes in void ratio Δe, incremental stress $\Delta\boldsymbol{\sigma}$, and incremental plastic strain $\Delta\boldsymbol{\epsilon}^{\mathbf{p}}$}.**(b)** Prediction accuracy under a triaxial loading path defined by $\xi = -1000$, $p_{\text{in}} = 375$ kPa, and $e_{\text{in}} = 0.64$. The absolute errors in predicting the mean effective stress (p) and deviatoric stress (q) are assessed by comparing model outputs to reference results obtained from high-accuracy numerical integration.**(c)** Training convergence trends for serial and parallel architectures. The cKAN sub-networks use relatively compact architectures, with 2 layers of 20 neurons (degree 3) for the void ratio, and 3 layers of 20 neurons (degree 4) for both stress and plastic strain outputs. Moreover, the MLP counterparts employ deeper and wider networks, with 2 layers of 45 neurons for the void ratio and 5 layers of 35 neurons for stress and plastic strain, without polynomial basis functions. All panels present new analysis based on Mostajeran and Faroughi (2024).

Table 2
A non-exhaustive list of recent studies on application-oriented advances in data-driven KANs, highlighting their architectures, basis functions, and targeted real-world tasks across scientific and engineering domains.

Method	Application Focus	Ref.
EPi-cKAN	Developed physics-guided serial and parallel architectures to model nonlinear elasto-plastic behavior of granular materials; Used Chebyshev polynomials as basis functions; Applied to predict stress-strain responses.	(Mostajeran & Faroughi, 2024)
Time-Series KAN	Developed a KAN framework for time series forecasting; Used B-spline basis functions (degree 3 and grid size 5); Applied to forecast real-world satellite traffic using normalized hourly data from the 5G-STARDUST project.	(Vaca-Rubio et al., 2024)
Geotechni-cal KAN	Developed a KAN model to predict settlement and residual strength after liquefaction; Used B-spline basis functions; Applied to field and lab data from case studies, including risk assessment in Patna and railway embankment analysis.	(Karakaş, 2024)
COEFF-KANs	Developed a two-stage modeling strategy to predict Coulombic Efficiency (CE) of lithium-metal battery electrolytes; Applied to a real-world Li/Cu half-cell dataset to predict log-transformed CE values.	(Li et al., 2024b)
SCKans-former	Developed a fine-grained classification model to improve diagnostic accuracy in hematological malignancies by addressing limitations of existing microimage analysis methods; Applied to over 10,000 cell images across three datasets (BMCD-FGCD, PBC, ALL-IDB).	(Chen et al., 2024b)
U-KAN	Developed a KAN-enhanced convolutional architecture to improve nonlinear representation in image segmentation and generative tasks; Used tokenized KAN layers; Applied to diverse imaging datasets (BUSI, GlaS, and others).	(Li et al., 2025)
KACQ-DCNN	Developed a classical-quantum hybrid neural architecture; Used B-spline basis functions within a dual-channel design to model nonlinear activation patterns and capture complex feature relationships; Applied to structured data for binary classification.	(Jahin et al., 2024)
KAN-IDIR	Developed an implicit neural representation framework for deformable image registration with KANs; Utilized adapted Chebyshev KANs ; Applied to three medical imaging benchmarks: DIR-Lab lung CT, OASIS-1 brain MRI, ACDC cardiac MRI.	(Drozdov et al., 2025)

Table 8
Comparative summary of major KAN developments in the context of methodological challenges across data-driven, physics-informed, and deep-operator learning paradigms.

Model Name	Accuracy	Convergence	Theoretical Analysis	Computational Analysis	Code	Ref
(I) Data-driven						
KAN	✓	✓	✓	✓		(Liu et al., 2024c)
Chebyshev KAN	✓	✓	×	×		(SS et al., 2024)
FastKAN (RBF)	✓	×	×	✓		(Li, 2024)
Wav-KAN	✓	×	×	×		(Bozorgasl & Chen, 2024)
(II) Physics-informed						
PIKAN	✓	✓	✓	✓		(Wang et al., 2025b)
SPIKAN	✓	✓	✓	✓		(Jacob et al., 2024)
Scaled-cPIKAN	✓	✓	✓	×		(Mostajeran & Faroughi, 2025)
KKANs	✓	✓	×	✓		(Toscano et al., 2025c)
(III) Deep-operators						
DeepOKAN	✓	✓	×	×		(Abueidda et al., 2025)
Hybrid Operators	✓	✓	×	×	×	(Kiyani et al., 2025b)

2. Zhang, Fan & Lu — RSS-KAN

DOI: https://doi.org/10.1016/j.neucom.2026.132796

The core idea

This paper's starting complaint is specifically about the B-spline curve used in the original KAN: B-splines are computed through a step-by-step recursive process that doesn't run efficiently on GPUs, and their shape is somewhat rigid (each one is just a shifted, identically-shaped bump). Their proposed fix, RSS-KAN, replaces the B-spline on every edge with a custom-built curve that blends three familiar, simple functions: a ReLU (which acts like a window, turning the curve on and off over a specific local range), a Sigmoid (a smooth S-shaped curve that controls how the curve rises and levels off), and a Sine wave (which lets the curve wiggle periodically). Each edge learns its

own mixing weights for these three ingredients, so different edges can end up looking mostly ReLU-like, mostly smooth, mostly wavy, or some blend, depending on what the training data needs.

The motivation for choosing exactly these three: the ReLU part gives "local support" — the curve only does something over a bounded window, similar to how a B-spline works, which keeps changes in one part of the input space from messing with other parts. The Sigmoid part gives smooth shape control, letting the curve's peak height and steepness adapt. The Sine part lets the curve capture high-frequency, oscillating, or periodic patterns that a purely smooth curve (or a purely ReLU-based curve) would struggle to fit tightly.

all three of these ingredient functions can be computed using ordinary matrix operations — no recursion, no step-by-step evaluation like B-splines require. That's the paper's central claim: this hybrid curve keeps the flexibility advantage of a rich, multi-part basis function while staying fully GPU-friendly.

The paper also proves that this hybrid curve is strictly more expressive than using any one of its three ingredients alone — a network built only from ReLU, only from Sigmoid, or only from Sine can each be reproduced as a special case of RSS-KAN by just turning the other ingredients' weights to zero. So nothing is lost by using the combined version; it can only do at least as much as any single-ingredient version.

How well it works

The authors tested RSS-KAN against five other improved KAN variants (each of which had already tried some other fix to the B-spline problem): BSRBF-KAN (blends B-splines with bell-shaped RBF curves), Efficient-KAN, Fast-KAN (swaps B-splines for a simpler bell-curve family), ReLU-KAN (the direct predecessor, using ReLU alone), and SineKAN (using Sine alone). The comparison used eight test functions of varying character — smooth, oscillating, high-dimensional, and combinations of these.

Reported accuracy results: RSS-KAN had the lowest error on six of the eight test functions, and specifically won by a wide margin on the functions with strong oscillation or asymmetry — exactly the cases its Sine ingredient is meant to help with. On two of the smoothest test functions, a competing model (Efficient-KAN, which uses smooth bell-shaped curves well-suited to gentle, structured functions) edged it out slightly. Averaged across the whole test set, the improvements over the other models ranged from roughly one order of magnitude (10x) up to three to five orders of magnitude (thousands to hundreds of thousands of times lower error), depending on which competitor and which test function.

Reported speed results: the original recursive B-spline KAN was by far the slowest model tested, as expected. RSS-KAN was not the fastest model overall — it was roughly twice as slow as the fastest competitors (Fast-KAN and ReLU-KAN) per training step, because blending three ingredients costs more than using just one. But it was in the same speed range as BSRBF-KAN and Efficient-KAN, and still 5 to 15 times faster overall than the original B-spline KAN. The paper argues that because RSS-KAN needs fewer training steps to reach a given accuracy, its total training time ends up competitive with the fastest models even though each individual step costs a bit more.

Experiment results

A component-removal (ablation) study tested every combination of the three ingredients alone and in pairs, to check whether the full three-way blend is actually necessary or if it's just extra complexity. Using only ReLU worked fine on functions with sharp corners but did poorly on periodic functions. Using only Sigmoid worked for smooth, gently-curving functions but struggled with local oscillation. Using only Sine was excellent for periodic functions but converged slowly on non-periodic ones. Every two-ingredient pairing did better than either ingredient alone, and the full three-ingredient version did best across the board — supporting the idea that the three pieces genuinely complement rather than duplicate each other. The authors also tried adding a fourth ingredient (Tanh) on top of all three, which did not help and sometimes hurt performance slightly, suggesting three well-chosen ingredients is close to the sweet spot rather than "more is always better."

The paper also tracked how the mixing weights for the three ingredients evolve during training, and found the network tends to quickly figure out which ingredient should dominate for a given task (e.g., leaning hard into Sigmoid for a smooth transition, or into Sine for a periodic pattern) in the early stages of training, then fine-tunes the balance later as the model refines its fit. Deliberately removing or perturbing one ingredient's contribution after

training showed that the "dominant" ingredient for a given task mattered the most, but the other two still played a real, measurable supporting role rather than being dead weight.

Real-world test

Beyond the synthetic test functions, the authors tried RSS-KAN on a genuine weather dataset, predicting air temperature from other measured weather variables over a 24-hour sliding window. RSS-KAN produced lower training and validation error than every comparison model, including a standard MLP, and its predicted temperature curve tracked the true curve more closely, especially around the daily peaks and troughs where competing models tended to lag behind or overshoot.

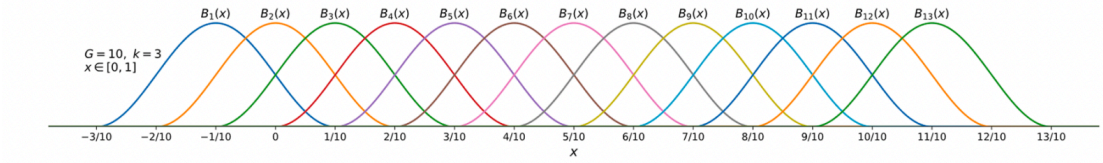


Fig. 1. B-splines with $G = 10$ and $k = 3$.

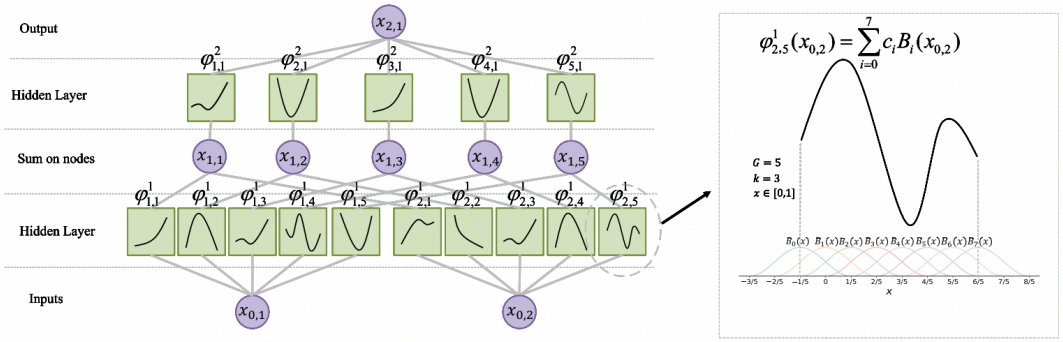


Fig. 2. Example of the B-splines fitting process.

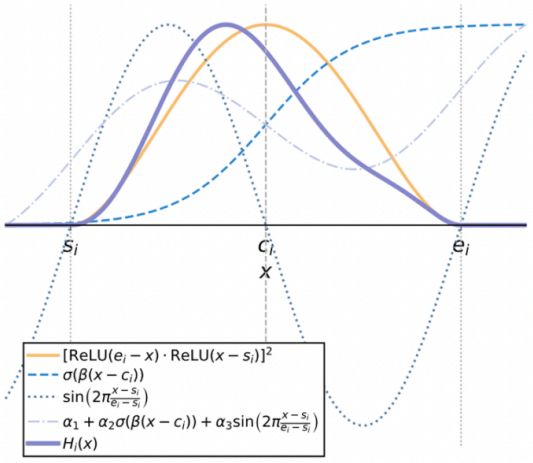
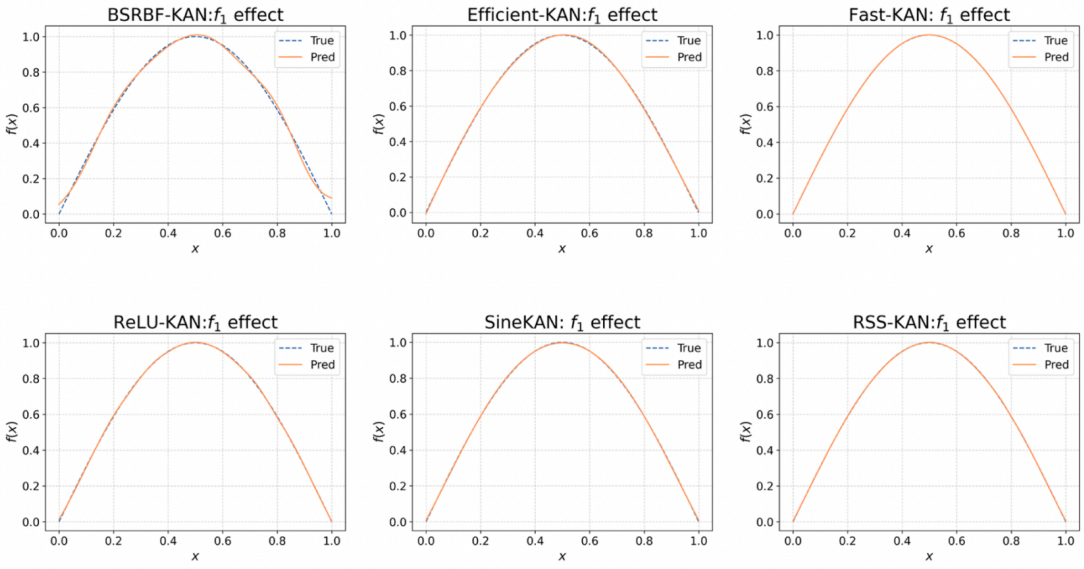
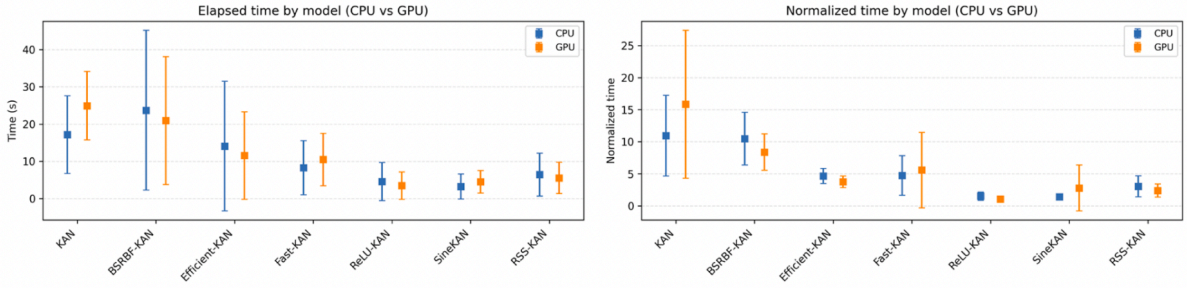


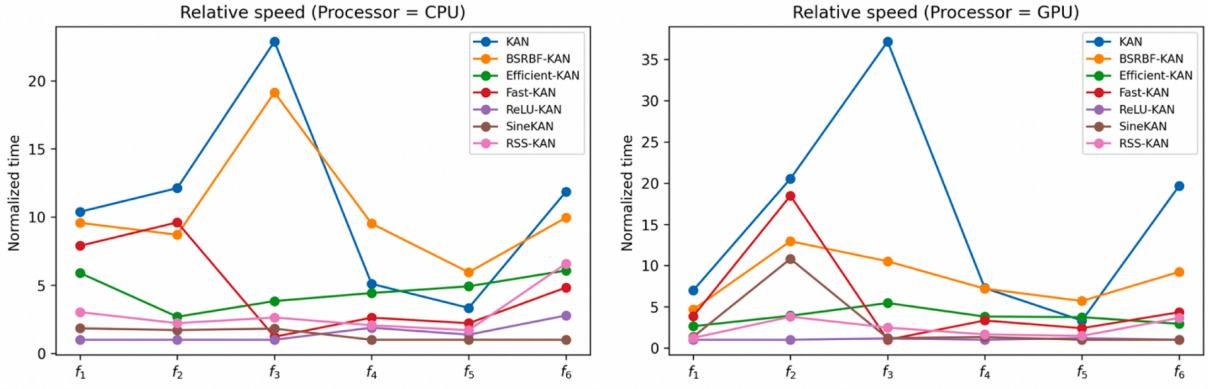
Fig. 3. Basis function$H_i\ (x)$and its component function plots.



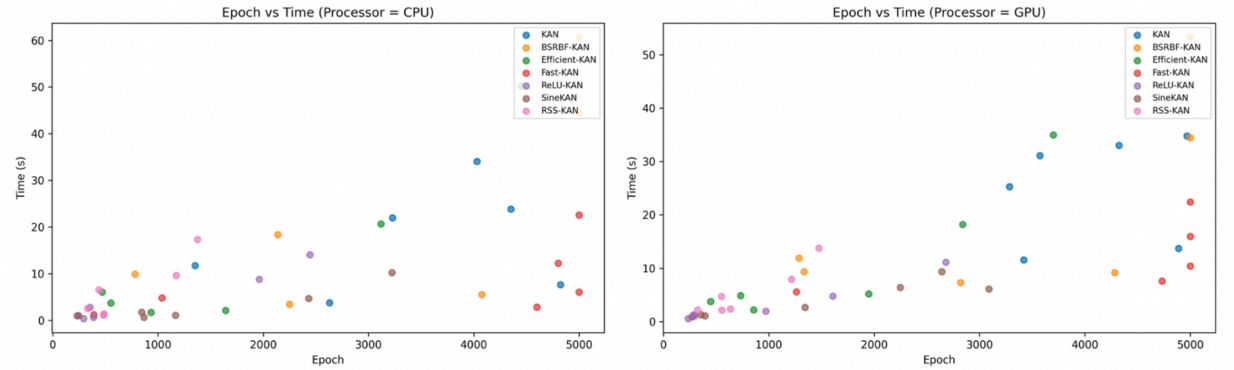
(a) Fitting curves of the f_1 function for different models. Model parameter settings: BSRBF-KAN model: shape=[1,1], G=5, k=3; Efficient-KAN: shape=[1,5,1], G=5, k=3; Fast-KAN: shape=[1,5,1], G=5, k=3; ReLU-KAN: shape=[1,1], G=5, k=3; SineKAN model: shape=[1,1], G=5, k=3; RSS-KAN: shape=[1,1], G=5, k=3.



(a) Training time comparison between CPU and GPU. The left figure shows the time required for different models to achieve MSE=0.001 when fitting six functions. The right figure indicates the relative speed of different models when achieving MSE=0.001 while fitting six functions.



(b) Relative speed line chart of different models on various processors when fitting six functions to achieve MSE=0.001.



(c) The relationship between the number of training iterations and time required for different models to achieve an MSE of 0.001 when fitting six functions on different processors.

Fig. 6. Comparison of speed performance among different models at the same target MSE. Parameter settings for the six models: When fittingf_3, f_4, BSRBF-KAN model: shape= [2, 8, 1], G= 5, k = 3; Fast-KAN model: shape= [2, 6, 1], G= 5, k = 3; When fittingf_5, BSRBF-KAN model: shape= [2, 6, 3, 1], G= 10, k = 3; Fast-KAN model: shape= [2, 5, 2, 1], G= 10, k = 3; When fittingf_6, BSRBF-KAN model shape= [6, 12 ,6, 1], G= 10, k = 3; Fast-KAN model shape= [6, 10, 5, 1], G= 5, k = 3; Other model parameters are identical to those in Table 3.

3. Wolff, Alesiani, Duhme & Jiang — Clifford Kolmogorov-Arnold Networks

Link: https://arxiv.org/abs/2602.05977

The core idea

This paper generalizes KANs to work with a broader kind of number than the usual real or complex numbers. A prior paper by some of the same authors had already extended KANs to complex numbers (CVKAN), useful for problems like signal processing where quantities naturally have both a magnitude and a phase. This new paper goes a step further, extending KANs to Clifford algebras — a mathematical framework that generalizes complex numbers and quaternions into a much richer family of "multi-part" numbers capable of representing not just points, but also directions, rotations, areas, and volumes, all within one unified number system. The motivating examples given are things like electromagnetic fields, 3D geometry, and robot arm movement, where a single physical quantity naturally has several intertwined components that a plain real or complex number can't capture cleanly.

Structurally, the idea is straightforward given the CVKAN precedent: instead of each KAN edge learning a curve that maps a real number to a real number (or a complex number to a complex number), each edge in a Clifford-KAN learns a curve that maps one of these richer multi-part numbers to another multi-part number, letting the network learn transformations that respect the internal structure of the input (for example, correctly handling how a rotation and a direction interact, rather than treating each numeric component independently).

Each edge's curve is still built the same conceptual way as prior KAN variants: a set of bump-shaped basis functions (RBFs, the same bell-curve idea as FastKAN) placed on a grid, each with a learnable weight, and the curve's output is the weighted sum of these bumps. The novelty is that both the grid points and the weights now live in this richer Clifford-number space rather than being plain real numbers.

The main problem this paper had to solve: too many grid points

The natural way to build this grid — spacing points evenly across every dimension of the Clifford-number space — runs into a serious scaling problem. As you move to richer number systems (e.g., quaternions, which have four components instead of two like complex numbers), the number of grid points needed to cover the space evenly grows exponentially. The paper gives a concrete example: with a modest 8 points per dimension, a quaternion-based

network needs over 4,000 grid points, each with its own learnable weight, just for a single edge. This is described as the central obstacle to making higher-dimensional Clifford-KANs practical.

Their proposed fix is to abandon the evenly-spaced grid entirely and instead scatter the grid points using a particular kind of structured randomness (a "Sobol sequence," a well-known technique from numerical simulation for spreading sample points evenly through a space without needing exponentially many of them). The paper shows that this Sobol-based grid gives an unbiased approximation of the same target that the full evenly-spaced grid would give, and that its accuracy improves in a predictable, well-behaved way as more points are added — without ever needing the full exponential grid size.

Normalization: keeping the network well-behaved during training

Because the network's grid only covers a fixed range, the paper also had to make sure values flowing between layers don't drift outside that range and end up far from every grid point (which would make the learned curve useless for those inputs). Three different ways of normalizing the data between layers were tried: normalizing each individual component of the multi-part number across the whole layer, normalizing each full multi-part number as a unit, or normalizing each component of each individual node separately. A fourth option — doing no normalization at all — was also tested as a baseline.

How well it worked

On the same simple test functions used to validate the earlier complex-number version (CVKAN), the new Clifford-KAN matched CVKAN's performance when using the full, evenly-spaced grid, confirming the extension didn't lose anything. The Sobol (randomly-scattered) grid's accuracy improved steadily as more points were added, and by around 8 points per dimension it essentially caught up to the full grid — while needing dramatically fewer parameters to get there in higher-dimensional settings.

On a larger, more realistic dataset (100,000 samples related to holography), the full-grid version of Clifford-KAN performed close to CVKAN, and a Sobol grid using only about three-quarters of the full grid's points achieved nearly identical accuracy — evidence that the random-grid trick doesn't cost much accuracy even on a real dataset, not just toy problems. Grids that were too sparse (very few points per dimension) failed to learn the function well, so there's a floor below which the savings stop being worth it.

When they tested genuinely higher-dimensional Clifford number systems (including a quaternion-like system and a couple of others used in computer vision and robotics), the Sobol grid generally matched the full grid's accuracy while using a small fraction of the parameters — in one specific case, matching or even beating the full grid's accuracy while using only about 2% of its parameter count. This is presented as the paper's strongest result: the parameter-saving trick doesn't just avoid hurting accuracy, it can sometimes help, though this benefit was inconsistent across every number system and dataset tested.

On the normalization comparison, doing no normalization at all was clearly the worst option by a wide margin, while the three actual normalization methods all performed similarly to each other — the paper's conclusion is that some form of normalization is essential, but which specific type you pick doesn't matter very much.

Stated limitations and open issues

- Both CVKAN and the new Clifford-KAN were reported to be quite sensitive to how the network's weights are initialized before training; the authors had to increase the learning rate substantially compared to the earlier CVKAN paper to get more consistent, competitive results, and note that some instability across different training runs still remains even with this fix.
- One of the four test functions (a function with especially large swings in its output values) was reported to be far harder to fit well than the others, for both models.
- The random Sobol grid didn't always win — for some specific combinations of number system, dataset, and grid density, it performed worse than the full grid, with notably high run-to-run variability in those cases. The authors describe this as an open, not fully resolved, practical inconsistency rather than a clean win in every setting.

Model	Norm	Test MSE	Test MAE
CVKAN	nodew.	0.016 ±0.001	0.066±0.003
	dimw.	0.015 ±0.003	0.062±0.008
	compw.	0.017 ±0.002	0.063±0.004
	no	51.059 ±62.526	3.278±3.951
$\mathbb{C}l$KAN	nodew.	0.025 ±0.004	0.069±0.006
	dimw.	0.021 ±0.003	0.060±0.005
	compw.	0.028 ±0.009	0.071±0.006
	no	0.867 ±1.061	0.354±0.382

4. Optimized Architectures for Kolmogorov–Arnold Networks

Citation: James Bagrow and Josh Bongard, "Optimized Architectures for Kolmogorov-Arnold Networks," Apr 20, 2026.

Link: https://arxiv.org/abs/2512.12448

Core idea

The paper addresses a stated tension: architectural enhancements that improve KAN accuracy (skip connections, added depth, DenseNet-style blocks) tend to reduce the interpretability that motivates using KANs in the first place. The authors propose training an overprovisioned KAN together with a differentiable sparsification and depth-selection mechanism, so that a compact, interpretable model is discovered during training rather than chosen by hand.

Three architectural mechanisms are combined:

- Edge gates ("egates") — a differentiable relaxation of $\ell 0$ regularization (following Louizos et al., 2018) applied per activation-function edge, allowing individual learnable spline activations to be pruned to exactly zero during training.
- DenseNet-style forward connections ("FCs") — each layer receives the concatenation of all previous layer outputs (and the original input), providing deep supervision and letting the network skip unnecessary depth.
- Multi-exit / exit gates ("xgates") — every layer gets its own output head; a differentiable categorical (Gumbel-Softmax) gate learns which exit to use as the final output, providing explicit depth selection.

Training is guided by a minimum description length (MDL) objective that jointly penalizes prediction error and the number of "open" gates (edges, nodes, and the active exit), following a BIC-style approximation for model complexity.

Reported results

The authors ran a 2×2×2 factorial ablation (egates present/absent × forward connections present/absent × exit gates present/absent) across three problem classes: symbolic function-approximation benchmarks (the first 10 Nguyen problems), two dynamical-systems forecasting tasks (the Ikeda map and a 3-species ecosystem model), and two real-world regression datasets (concrete compressive strength, superconductor critical temperature).

- Edge gates alone ("E") were reported as necessary but not sufficient: condition E reduced model size but consistently harmed accuracy relative to baseline across the Nguyen problems ("E led to worse accuracy than baseline on all problems").

- Conditions combining edge gates with a depth-selection mechanism — EF, EX, or EFX — were reported as the best performers, generally producing models that were both smaller and more accurate than the standard KAN baseline.
- On the Ikeda map, the EX condition matched baseline accuracy using 16 contributing edges versus 48 for baseline, per the paper's supplementary tables.
- On the concrete-strength dataset, the EFX condition at one regularization setting (β = 0.01) was reported to achieve 4.87 MPa RMSE versus the baseline's 4.91 MPa, using a model with 64 contributing edges versus 351 for baseline (about 18% of the size).
- A Pareto hypervolume analysis (trading off RMSE, number of contributing edges, and compositional depth) ranked condition EX highest on average across the five problem domains, with EF and EFX also ranking well; no single condition dominated on every domain.

The authors also list an explicit limitation: their differentiable gating mechanism is noted to have known variance issues during training, and the ecosystem dynamical-systems task "was prone to overregularization," indicating the method is not push-button and needs hyperparameter tuning of the sparsity weight β and initialization values.

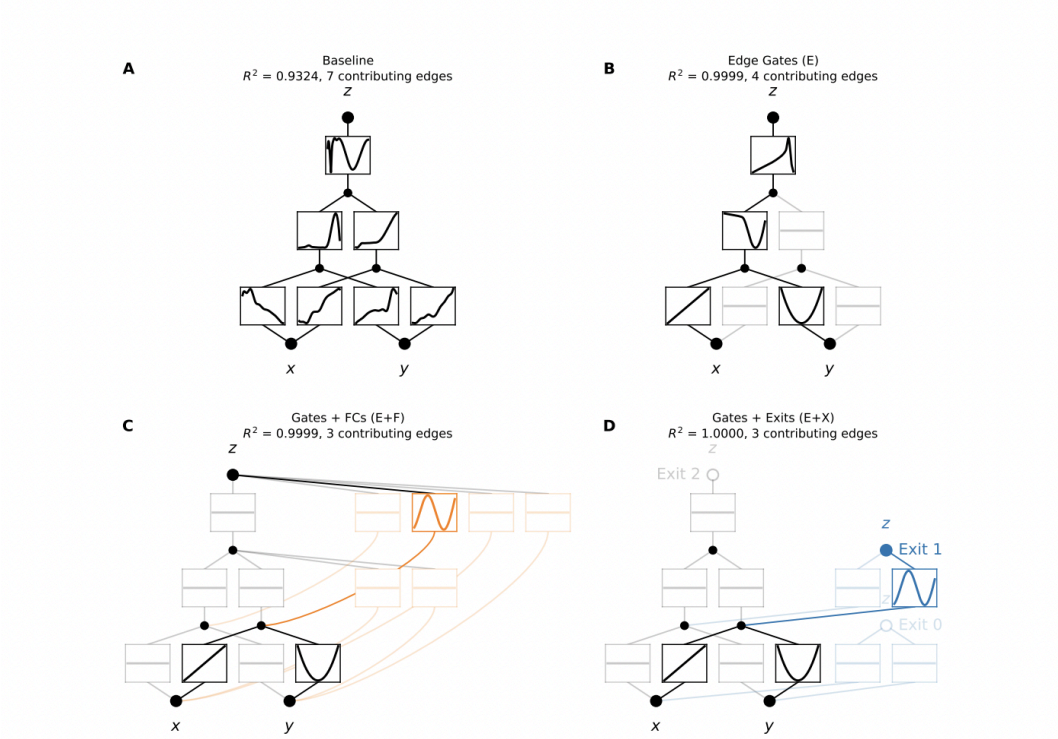


Figure 1: Learning $z = \sin(x + y^2)$ with overprovisioned KANs (architecture $[2, 2, 1, 1]$). Forward connections are shown in orange and exit gates in blue.

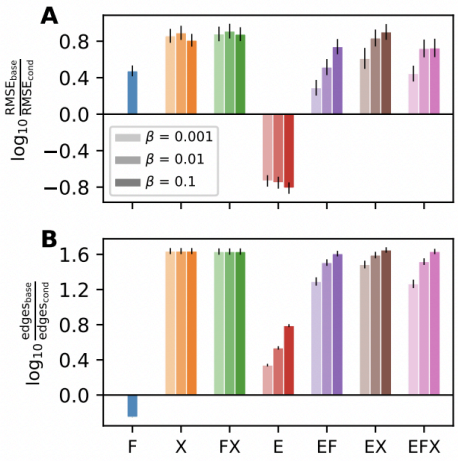


Figure 2: Model accuracy and size relative to KAN baseline. Log-ratios of (**A**) test RMSE and (**B**) contributing edges, pooled across Nguyen problems F1–F10 (see also App. B) and broken down by condition and β. Positive values indicate conditions have lower RMSE or fewer contributing edges than baseline. Errorbars denote ±1 SEM on log-ratios.